

Computational Complexity Overview (S-learner)

1 Complexity Table

In Table 1, we present the general computational complexity, independent of any specific model, using an S-learner for *SCOPE* and SEP. In Table 2, we detail the computational complexity when using XGBoost (in an S-learner), the main model used in the experiments. Assuming the exact greedy algorithm for split finding in XGBoost, we approximate $T_{infer}(f) = \mathcal{O}(E \cdot D)$ per sample, and $T_{train}(n, f) = \mathcal{O}(E \cdot D \cdot n \cdot f \log n)$ for n samples, where E is the number of trees and D is the maximum tree depth.

Method	Training (Offline)	Inference (Online, per case)
SCOPE-S	$\mathcal{O}(K \cdot [T_{train}(C, d_s + A) + C \cdot A \cdot T_{infer}(d_s + A)])$	$\mathcal{O}(K \cdot A \cdot T_{infer}(d_s + A))$
SEP-S	$\mathcal{O}(K \cdot T_{train}(C, d_s + A))$	$\mathcal{O}(K \cdot A \cdot T_{infer}(d_s + A))$
Single Model	$\mathcal{O}(T_{train}(K \cdot C, d_s + K \cdot A))$	$\mathcal{O}(K \cdot A \cdot T_{infer}(d_s + K \cdot A))$
KMeans-Q	$\mathcal{O}(K \cdot C \cdot I \cdot k_c \cdot d_s)$	$\mathcal{O}(K \cdot k_c \cdot d_s)$

Table 1: Computational complexity of training and inference.

Method	Training (Offline)	Inference (Online, per case)
SCOPE-S	$\mathcal{O}(K \cdot C \cdot E \cdot D \cdot ((d_s + A) \log C + A))$	$\mathcal{O}(K \cdot A \cdot E \cdot D)$
SEP-S	$\mathcal{O}(K \cdot C \cdot E \cdot D \cdot (d_s + A) \log C)$	$\mathcal{O}(K \cdot A \cdot E \cdot D)$
Single Model	$\mathcal{O}(K \cdot C \cdot E \cdot D \cdot (d_s + K \cdot A) \log(C \cdot K))$	$\mathcal{O}(K \cdot A \cdot E \cdot D)$
KMeans-Q	$\mathcal{O}(K \cdot C \cdot I \cdot k_c \cdot d_s)$	$\mathcal{O}(K \cdot k_c \cdot d_s)$

Table 2: Computational complexity of training and inference assuming an XGBoost model.

Legend

C : cases K : max decisions per case A : max action space size d_s : prefix feature dimension
 $T_{train}(n, f)$: training time on n samples, f features $T_{infer}(f)$: inference time per sample k_c : number of clusters
 I : KMeans iterations E : number of estimators D : tree depth

2 Complexity Derivations

SCOPE-S

The algorithm iterates backwards through K decision points. At each step, it trains a regression model on at most C samples. To compute the regret-based value updates for the previous step, it must perform inference on the newly trained model using all A possible actions for all C cases, adding an inference overhead of $\mathcal{O}(C \cdot A \cdot T_{infer})$.

SEP-S (de Leoni et al., adjusted implementation for fair comparison [2])

This method trains K independent regression models, one for each decision point. Each model is trained on a dataset of at most C cases. Because it does not account for future actions (no backward induction), it entirely avoids the $\mathcal{O}(C \cdot A)$ action-search bottleneck during training.

Single Model (de Leoni et al., original implementation [2])

Instead of training multiple step-specific models, this method pools every observed state-action pair across all cases and decision points into a single dataset of size $C \cdot K$. The complexity heavily depends on the action space encoding. In the worst case, if actions are highly specific to each decision point, the feature space may expand up to $d_s + K \cdot A$ to accommodate all distinct actions. However, if the decision

points share a common pool of actions, the proposed action can be encoded globally, keeping the feature space constrained to $d_s + A$. The model then trains globally to predict the final KPI.

KMeans-Q (Branchi et al. [1])

Training consists of two sequential phases: clustering and Q-learning. Clustering all $C \cdot K$ prefixes takes $\mathcal{O}(C \cdot K \cdot I \cdot k_c \cdot d_s)$. Running tabular Q-learning for a maximum of C episodes (where each episode is up to K steps long) takes $\mathcal{O}(C \cdot K)$. The Q-learning complexity is absorbed by the strictly larger clustering complexity in Big-O notation. During inference, it calculates the distance to k_c clusters and performs a constant-time table lookup.

3 Analysis and Discussion

Training Complexity

When compared to the baseline methods, *SCOPE-S* introduces additional computational requirements to achieve its sequential policy. It is strictly more computationally demanding than SEP-S, which trains independent models and omits the $\mathcal{O}(C \cdot A \cdot T_{infer})$ backward induction term entirely. Compared to the Single Model approach, which pools data to train a single regression model on $C \cdot K$ samples, the relative efficiency of *SCOPE-S* depends on the scaling properties of the chosen regression algorithm (e.g., whether it scales linearly or quadratically with sample size and feature dimension). Finally, KMeans-Q circumvents regression entirely in favor of clustering, operating in $\mathcal{O}(K \cdot C \cdot I \cdot k_c \cdot d_s)$. Overall, KMeans-Q is generally the most efficient method to train as it scales linearly with the dataset size and feature space, assuming a constant number of clusters (k_c) and iterations (I).

When specifically employing XGBoost (as detailed in Table 2), the training complexities become dependent on the number of trees E , maximum tree depth D , and a logarithmic factor of the dataset size introduced by the exact greedy split-finding algorithm. In this context, the training overhead of *SCOPE-S* compared to SEP-S is quantified by the additional additive term $\mathcal{O}(K \cdot C \cdot E \cdot D \cdot A)$, reflecting the inference passes required for backward induction. Furthermore, for the Single Model, pooling all data changes the logarithmic factor from $\log(C)$ to $\log(C \cdot K)$, while the worst-case action space expansion inflates the linear feature term from $(d_s + A)$ to $(d_s + K \cdot A)$.

Inference Complexity

SCOPE-S has an inference complexity that is identical to SEP-S. When compared to the Single Model, *SCOPE-S* is either equivalent or more efficient; if the Single Model requires an expanded feature space $(d_s + K \cdot A)$ to account for non-overlapping actions across different decision points, its inference cost (T_{infer}) will increase accordingly. KMeans-Q operates under a different paradigm during inference, relying on a centroid distance calculation and a table lookup bounded by $\mathcal{O}(K \cdot k_c \cdot d_s)$. Consequently, KMeans-Q is the most efficient method at runtime, as it scales strictly with the state dimensionality and cluster count while completely avoiding the action space multiplier (A) and predictive model execution (T_{infer}). *SCOPE-S* and SEP-S share the position of being the most efficient regression-based methods during inference.

When utilizing XGBoost, inference complexity depends primarily on the number of trees and their depth, bounded by $\mathcal{O}(E \cdot D)$. Notably, traversing a decision tree is independent of the overall feature space dimensionality (d_s and A), meaning T_{infer} stays constant regardless of how wide the input feature vector is. In this scenario, the inference advantage of *SCOPE-S* and SEP-S over the Single Model is neutralized in terms of Big-O notation, as all three evaluate the same number of trees. The main distinction remains between the regression approaches and KMeans-Q, which relies on centroid distance calculations rather than tree traversal.

4 Practical Runtime

We also evaluated the practical runtimes for our tested methods: *SCOPE-S*, SEP-S, and KMeans-Q. The original Single Model from [2] is omitted here (and in the paper), as SEP-S serves as our adapted implementation of it. Both *SCOPE-S* and SEP-S use XGBoost here.

Figure 1 illustrates the training times for each method on the SimBank dataset (varying training set sizes) and the SimBPIC17 dataset (varying the number of decision points). Interestingly, while

KMeans-Q has the most efficient theoretical complexity, it required the longest training time in practice. Profiling revealed that the KMeans clustering step was the primary bottleneck—accounting for 44 out of 46 seconds at a training size of 50K. This delay is potentially due to the large number of clusters resulting from the hyperparameter tuning (mostly in SimBPIC17, where k_c is around 5-140; in SimBank this is only around 30-80), combined with the number of iterations needed for the KMeans algorithm. *SCOPE-S* was the second most time-consuming method to train, primarily due to the additional inference required for backward induction.

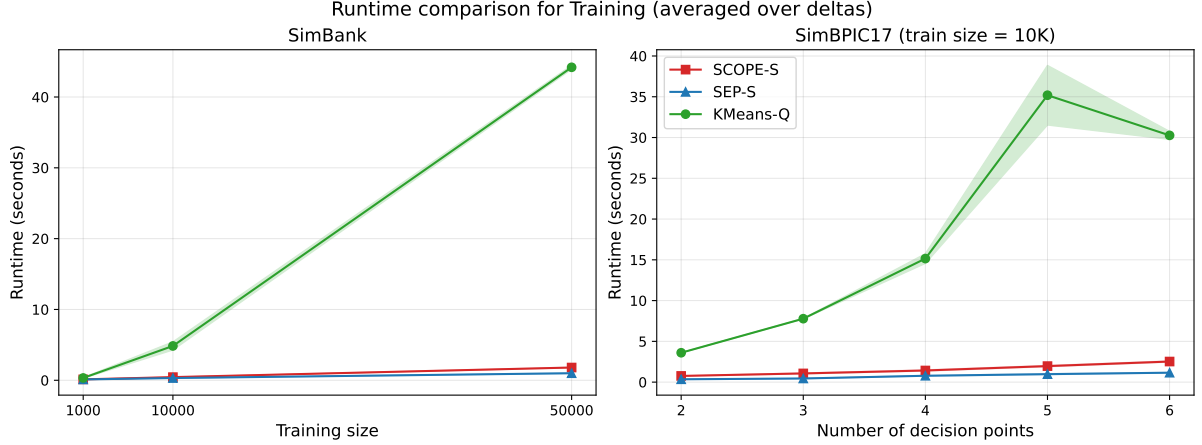


Figure 1: Runtime in seconds for training, averaged over all concerned confounding levels in the paper and iterations. *SCOPE* and Sep: XGBoost, S-learner.

Figure 2 compares the inference times. *SCOPE-S* generally exhibits the highest inference latency, except at larger numbers of decision points in SimBPIC17 where KMeans-Q becomes slower. This crossover likely occurs again because of the large number of clusters in SimBPIC17, and potentially because KMeans-Q’s theoretical advantage of not having a multiplier A diminishes when A is relatively small (e.g., for SimBPIC17 $A = 2$). Finally, while *SCOPE-S* and SEP-S share the same theoretical complexity, *SCOPE-S* exhibited a longer practical runtime. As shown in Figure 3, this is simply caused by the underlying models in *SCOPE-S* utilizing a greater combined tree count and depth ($E \cdot D$), which directly impacts inference time according to the complexity tables. Since the tuning process was identical for both methods, this increased model complexity is possibly a consequence of *SCOPE-S*’s backward induction step. Because *SCOPE-S* predicts a compounded and regret-based target variable rather than the raw KPI (as used in SEP-S), it potentially requires more expressive models, though further research is needed to confirm this.

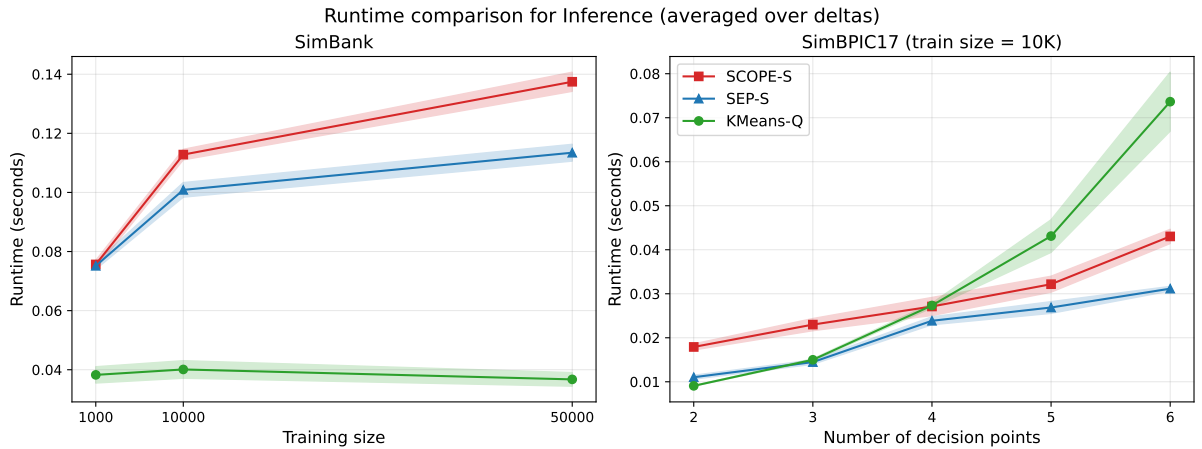


Figure 2: Runtime in seconds for inference (10K test cases for SimBank, 1K for SimBPIC17), averaged over all concerned confounding levels in the paper and iterations. *SCOPE* and Sep: XGBoost, S-learner.

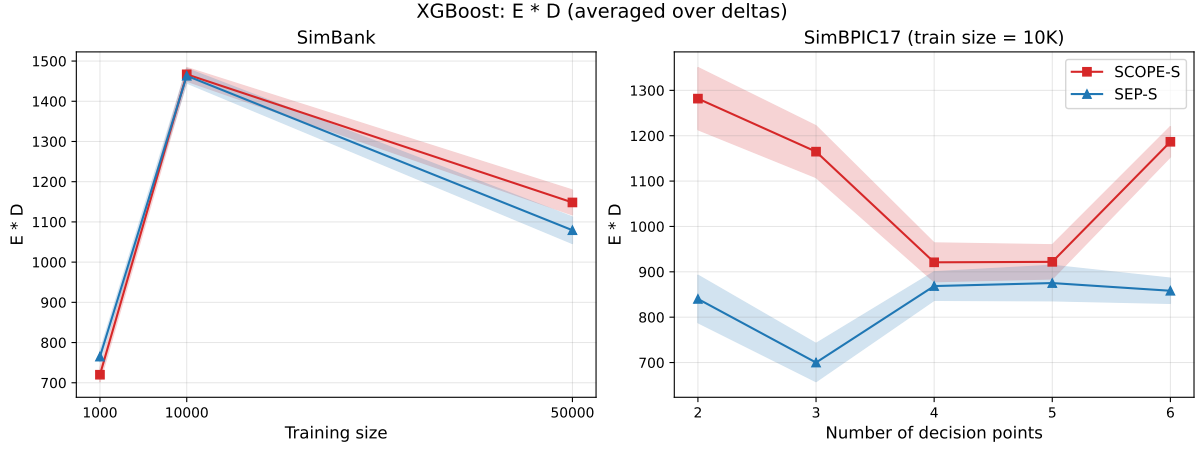


Figure 3: The number of trees (E) times the average depth of a tree (D) for *SCOPE* and Sep.

References

- [1] Branchi, S., Buliga, A., Francescomarino, C.D., Ghidini, C., Meneghello, F., Ronzani, M.: Recommending the optimal policy by learning to act from temporal data (2023)
- [2] Leoni, M.d., Dees, M., Reulink, L.: Design and evaluation of a process-aware recommender system based on prescriptive analytics. In: ICPM (2020)